

Lab Sheet: 2

Objective:

- To understand and implement SQL transactions using COMMIT, SAVEPOINT, and ROLLBACK, retrieve records using SELECT queries, and perform INSERT, UPDATE, and ALTER operations on tables.

Instructions:

1. Open your preferred Database Management System (DBMS) tool.
2. Create the required tables: CLASS, EMPLOYEES, and Client_master.
3. Insert the given/suitable records into each table.
4. Execute the SQL scripts and queries step by step.
5. Observe the output carefully after each operation, especially after ROLLBACK and UPDATE commands.

Table Structures:

CLASS

D (NUMBER) and Name (VARCHAR).

The initial records in the table are (1, 'Bravo'), (2, 'Alex'), and (4, 'Cheng').

EMPLOYEES

Employee_Id (NUMBER), First_Name (VARCHAR), Last_Name (VARCHAR), Email (VARCHAR), Phone_Number (VARCHAR), Hire_Date (DATE), Job_Id (VARCHAR), Salary (NUMBER), Commission_Pct (NUMBER), Manager_Id (NUMBER), and Department_Id (NUMBER).

DDL Questions (Data Definition Language):

1. How do you create a table named CLASS with appropriate data types?
2. How do you create a table named EMPLOYEES with the specified attributes?
3. How do you create a table named Client_master with the specified attributes?
4. What SQL statement would you use to rename the table Client_master to Client12?
5. How do you display the bal_due column heading as "BALANCE"?

DML Questions (Data Manipulation Language):

Part A: Transactions and Savepoints (CLASS Table)

Give the output of the following SQL script:

- INSERT INTO class VALUES (5,'Rahul');
- COMMIT;
- UPDATE class SET name = 'Abhijeet' WHERE id='5';
- SAVEPOINT A;
- INSERT INTO class VALUES (6,'Chris');
- SAVEPOINT B;
- INSERT INTO class VALUES (7,'Bravo');
- SAVEPOINT C;
- SELECT * FROM class;
- ROLLBACK TO B;
- SELECT * FROM class;
- ROLLBACK TO A;

Part B: Retrieving Records (EMPLOYEES Table)

1. Find out the employee id, names, and salaries of all the employees.
2. List out the employees who work under manager 100.
3. Find the names of the employees who have a salary greater than or equal to 4800.

4. List out the employees whose last name is 'AUSTIN'.
5. Find the names of the employees who work in departments 60, 70, and 80.
6. Display the unique Manager_Id.

Part C: Updating Records (Client_master Table)

1. Insert five records into the Client_master table.
2. Find the names of clients whose bal_due > 5000.
3. Change the bal_due of ClientNO "C123" to Rs. 5100.

ANSWERS

DDL QUESTIONS:

Q1. How do you create a table named CLASS with appropriate data types?

```
CREATE TABLE CLASS (  
    Id NUMBER PRIMARY KEY,  
    Name VARCHAR2(50)  
);
```

Insert initial records:

```
INSERT INTO CLASS VALUES (1,'Bravo');  
INSERT INTO CLASS VALUES (2,'Alex');  
INSERT INTO CLASS VALUES (4,'Cheng');  
COMMIT;
```

Q2. How do you create a table named EMPLOYEES with the specified attributes?

```
CREATE TABLE EMPLOYEES (  
    Employee_Id NUMBER PRIMARY KEY,  
    First_Name VARCHAR2(50),  
    Last_Name VARCHAR2(50),  
    Email VARCHAR2(100),  
    Phone_Number VARCHAR2(20),  
    Hire_Date DATE,  
    Job_Id VARCHAR2(20),  
    Salary NUMBER(10,2),  
    Commission_Pct NUMBER(5,2),  
    Manager_Id NUMBER,  
    Department_Id NUMBER  
);
```

Q3. How do you create a table named Client_master with the specified attributes?

```
CREATE TABLE Client_master (  
    ClientNo VARCHAR2(10) PRIMARY KEY,  
    Name VARCHAR2(50),  
    City VARCHAR2(50),  
    Pincode NUMBER,  
    State VARCHAR2(50),  
    Bal_due NUMBER(10,2)  
);
```

Q4. What SQL statement would you use to rename the table Client_master to Client12?

```
RENAME Client_master TO Client12;
```

Q5. How do you display the bal_due column heading as "BALANCE"?

```
SELECT Bal_due AS "BALANCE"  
FROM Client_master;
```

DML QUESTIONS:

PART A: Transactions and Savepoints (CLASS Table)

```
CREATE DATABASE IF NOT EXISTS DBMS_Lab2_PartA;
USE DBMS_Lab2_PartA;

DROP TABLE IF EXISTS CLASS;
CREATE TABLE CLASS (
    Id INT PRIMARY KEY,
    Name VARCHAR(50)
);

INSERT INTO CLASS VALUES (1,'Bravo');
INSERT INTO CLASS VALUES (2,'Alex');
INSERT INTO CLASS VALUES (4,'Cheng');
COMMIT;

SET autocommit = 0;

INSERT INTO CLASS VALUES (5,'Rahul');
COMMIT;

UPDATE CLASS SET Name='Abhijeet' WHERE Id=5;
SAVEPOINT A;

INSERT INTO CLASS VALUES (6,'Chris');
SAVEPOINT B;

INSERT INTO CLASS VALUES (7,'Bravo');
SAVEPOINT C;

SELECT 'After All Inserts (C)' AS Stage, Id, Name FROM CLASS;

ROLLBACK TO B;
SELECT 'After ROLLBACK TO B' AS Stage, Id, Name FROM CLASS;

ROLLBACK TO A;
SELECT 'After ROLLBACK TO A' AS Stage, Id, Name FROM CLASS;

COMMIT;
```

Output:

Stage	Id	Name
After All Inserts (C)	1	Bravo
After All Inserts (C)	2	Alex
After All Inserts (C)	4	Cheng
After All Inserts (C)	5	Abhijeet
After All Inserts (C)	6	Chris

Result 1 x Read Only

Stage	Id	Name
After ROLLBACK TO B	1	Bravo
After ROLLBACK TO B	2	Alex
After ROLLBACK TO B	4	Cheng
After ROLLBACK TO B	5	Abhijeet
After ROLLBACK TO B	6	Chris

Result 2 x Read Only

Stage	Id	Name
After ROLLBACK TO A	1	Bravo
After ROLLBACK TO A	2	Alex
After ROLLBACK TO A	4	Cheng
After ROLLBACK TO A	5	Abhijeet

Result 3 x Read Only

PART B – RETRIEVING RECORDS (EMPLOYEES Table)

```
CREATE DATABASE IF NOT EXISTS DBMS_Lab2_PartB;
```

```
USE DBMS_Lab2_PartB;
```

```
DROP TABLE IF EXISTS EMPLOYEES;
```

```
CREATE TABLE EMPLOYEES (
    Employee_Id INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
```

```

Email VARCHAR(100),
Phone_Number VARCHAR(20),
Hire_Date DATE,
Job_Id VARCHAR(20),
Salary DECIMAL(10,2),
Commission_Pct DECIMAL(5,2),
Manager_Id INT,
Department_Id INT
);

INSERT INTO EMPLOYEES VALUES
(101,'John','Austin','john@gmail.com','9999999999','2020-01-10','IT_PROG',5000,NULL,100,60),
(102,'Neha','Sharma','neha@gmail.com','8888888888','2021-02-15','HR_REP',4500,NULL,100,70),
(103,'Rahul','Verma','rahul@gmail.com','7777777777','2019-03-20','IT_PROG',6000,NULL,101,80),
(104,'Anjali','AUSTIN','anjali@gmail.com','6666666666','2022-04-12','SA_REP',4800,NULL,100,60);

COMMIT;

```

1. Find out the employee id, names, and salaries of all the employees.

```
SELECT Employee_Id, First_Name, Last_Name, Salary FROM EMPLOYEES;
```

Employee_Id	First_Name	Last_Name	Salary
101	John	Austin	5000.00
102	Neha	Sharma	4500.00
103	Rahul	Verma	6000.00
104	Anjali	AUSTIN	4800.00
NULL	NULL	NULL	NULL

2. List out the employees who work under manager 100.

```
SELECT First_Name, Last_Name FROM EMPLOYEES WHERE Manager_Id = 100;
```

First_Name	Last_Name
John	Austin
Neha	Sharma
Anjali	AUSTIN

3. Find the names of the employees who have a salary greater than or equal to 4800.

```
SELECT First_Name, Last_Name FROM EMPLOYEES WHERE Salary >= 4800;
```

First_Name	Last_Name
John	Austin
Rahul	Verma
Anjali	AUSTIN

4. List out the employees whose last name is 'AUSTIN'.

5. `SELECT First_Name, Last_Name FROM EMPLOYEES WHERE Last_Name = 'AUSTIN';`

The screenshot shows a SQL query result grid with two columns: First_Name and Last_Name. The results are John Austin and Anjali AUSTIN. The interface includes a 'Filter Rows' field, 'Export' and 'Wrap Cell Content' buttons, and a 'Result Grid' button. The status bar at the bottom indicates 'EMPLOYEES 7' and 'Read Only'.

First_Name	Last_Name
John	Austin
Anjali	AUSTIN

5. Find the names of the employees who work in departments 60, 70, and 80.

`SELECT First_Name, Last_Name FROM EMPLOYEES WHERE Department_Id IN (60,70,80);`

The screenshot shows a SQL query result grid with two columns: First_Name and Last_Name. The results are John Austin, Neha Sharma, Rahul Verma, and Anjali AUSTIN. The interface includes a 'Filter Rows' field, 'Export' and 'Wrap Cell Content' buttons, and a 'Result Grid' button. The status bar at the bottom indicates 'EMPLOYEES 8' and 'Read Only'.

First_Name	Last_Name
John	Austin
Neha	Sharma
Rahul	Verma
Anjali	AUSTIN

6. Display the unique Manager_Id. give for part b answers.

7. `SELECT DISTINCT Manager_Id FROM EMPLOYEES;`

The screenshot shows a SQL query result grid with one column: Manager_Id. The results are 100 and 101. The interface includes a 'Filter Rows' field, 'Export' and 'Wrap Cell Content' buttons, and a 'Result Grid' button. The status bar at the bottom indicates 'EMPLOYEES 9' and 'Read Only'.

Manager_Id
100
101

PART C – CLIENT_MASTER TABLE

```
CREATE DATABASE IF NOT EXISTS DBMS_Lab2_PartC;  
USE DBMS_Lab2_PartC;
```

```
DROP TABLE IF EXISTS Client_master;
```

```
CREATE TABLE Client_master (  
    ClientNo VARCHAR(10) PRIMARY KEY,  
    Name VARCHAR(50),  
    City VARCHAR(50),  
    Pincode INT,  
    State VARCHAR(50),  
    Bal_due DECIMAL(10,2)  
);
```


1. Insert five records into the Client_master table.

```
INSERT INTO Client_master VALUES
('C123','Amit','Delhi',110001,'Delhi',5000),
('C124','Neha','Mumbai',400001,'Maharashtra',8000),
('C125','Rahul','Chennai',600001,'Tamil Nadu',3000),
('C126','Sneha','Kolkata',700001,'West Bengal',9000),
('C127','Kiran','Bangalore',560001,'Karnataka',2000);

COMMIT;
```

2. Find the names of clients whose bal_due > 5000.

```
SELECT Name, Bal_due FROM Client_master WHERE Bal_due > 5000;
```

Name	Bal_due
Neha	8000.00
Sneha	9000.00

3. Change the bal_due of ClientNO "C123" to Rs. 5100. gave answers for part c.

```
UPDATE Client_master SET Bal_due = 5100 WHERE ClientNo = 'C123';
COMMIT;
```

```
SELECT * FROM Client_master;
```

ClientNo	Name	City	Pincode	State	Bal_due
C123	Amit	Delhi	110001	Delhi	5100.00
C124	Neha	Mumbai	400001	Maharashtra	8000.00
C125	Rahul	Chennai	600001	Tamil Nadu	3000.00
C126	Sneha	Kolkata	700001	West Bengal	9000.00
C127	Kiran	Bangalore	560001	Karnataka	2000.00
* NULL	NULL	NULL	NULL	NULL	NULL